Desenvolvimento de Sistemas para Dispositivos Móveis

Corrotinas, Flow e ViewModel

DIM0524 — Turma 01 · Sprint 2 · aula de 26/10

Prof. Fernando · UFRN · 2026.2

A Sprint 2, a última

	Data
Conteúdo	hoje (passos 11 e 12) e em vídeo, até 30/10 (passos 13 a 16)
Acompanhamento online	09/11 e 16/11
Prova escrita	**11/11 (quarta)**, em laboratório
Daily meetings online	23 e 25/11 (presencial, se preferirem, em 18/11)
Entrega final	**30/11, 23:59**

Critério	Peso	Aula
Estado e arquitetura	25%	hoje e vídeo
Estados da interface	15%	hoje
Dados reais e erro de rede	25%	vídeo
Persistência local	10%	vídeo
Testes (interface e ViewModel)	15%	vídeo
App e documentação	10%	—

Onde vamos chegar

- O estado e as decisões saem da tela e vão para um ViewModel, que se testa sem abrir a tela
- As tarefas vêm da API de Web II, pela rede; sem rede, da última lista guardada
- Cada estado da tela (carregando, vazio, erro, sucesso) é um caso de uma classe selada

Branch `aula`**:** `./passo.sh 10` é onde a Sprint 1 terminou; hoje vamos até o 12.

Corrotinas em uma página

Já vistas na aula de Kotlin (`suspend`). O essencial para hoje:

Peça	O que faz
`suspend fun`	uma função que pode esperar (rede, disco) sem travar a thread
`launch { }`	inicia uma corrotina e segue em frente; devolve um `Job`
escopo (`CoroutineScope`)	dono das corrotinas: quando ele termina, as corrotinas dele são canceladas
`viewModelScope`	o escopo do ViewModel: cancelado quando o ViewModel deixa de existir
`Dispatchers.Main`	a thread da interface; as corrotinas do ViewModel começam nela

> Concorrência estruturada: toda corrotina tem um dono. Fechar a tela cancela o que ela pediu, sem vazamento.

📖 **Ref.** Android — Kotlin coroutines on Android

Por que a tela não pode esperar

- A tela é redesenhada a cada ~16 ms (60 quadros por segundo): tudo o que roda na thread principal disputa esse tempo
- **Bloquear** prende a thread até a resposta chegar; **suspender** libera a thread e continua a função depois, do mesmo ponto
- `suspend` não cria thread nem deixa nada mais rápido: só permite esperar sem prender

Concorrência estruturada

Situação	O que acontece
o usuário sai da tela antes da resposta	o ViewModel é descartado, o escopo é cancelado, e a busca para
uma corrotina filha falha	o erro sobe para o escopo, em vez de sumir em silêncio
o escopo é cancelado	cada corrotina recebe uma `CancellationException` no próximo ponto de suspensão

`GlobalScope.launch` não tem dono: ninguém cancela. No app, as corrotinas nascem no `viewModelScope`.

Em que thread a corrotina roda

Despachante	Para quê
`Dispatchers.Main`	atualizar a interface; é onde o `viewModelScope` começa
`Dispatchers.Default`	cálculo pesado, que usa CPU (ordenar milhares de itens, processar imagem)
`Dispatchers.IO`	chamadas que **bloqueiam** (arquivo com `java.io`, JDBC), só nos alvos JVM e nativos

- Funções `suspend` bem-feitas são seguras para chamar da Main: o Ktor Client e o DataStore trocam de thread por conta própria
- Por isso o exemplo não tem `withContext`: o ViewModel chama o repositório direto da Main
- No desktop, a Main é a thread do Swing: vem de `kotlinx-coroutines-swing`, acrescentada no Passo 11

Flow e StateFlow

	O que é	No app
`Flow<T>`	uma sequência de valores que chegam ao longo do tempo	a lista atualizada a cada mudança
`StateFlow<T>`	um `Flow` que sempre tem um valor atual, e emite quando ele muda	o estado da tela
`MutableStateFlow<T>`	o `StateFlow` que pode ser alterado	privado, dentro do ViewModel

- O estado é uma `data class` imutável: mudar é criar uma cópia com `copy`
- `update { }` faz a troca de forma segura, mesmo com várias corrotinas

📖 **Ref.** Android — StateFlow and SharedFlow

Flow frio, StateFlow quente

	Flow (frio)	StateFlow (quente)
quando roda	a cada `collect`, do começo	existe antes e depois dos coletores
valor atual	não tem	sempre tem (`.value`)
quem chega depois	recebe a sequência inteira	recebe o valor atual e as mudanças
valor igual ao anterior	é emitido	não é emitido

- Operadores (`map`, `filter`, `combine`) montam um `Flow` novo sem executar nada: a execução começa no `collect`
- Na Sprint 2, os `Flow` aparecem no DataStore (`data`) e nos testes com Turbine

Fluxo unidirecional de dados

- O estado desce; os eventos sobem. A tela nunca altera o estado direto
- É a elevação de estado da Sprint 1, um nível acima: quem guarda o estado agora é o ViewModel, e não uma função `@Composable`
- Para saber por que a tela está assim, basta olhar uma fonte: o `StateFlow`
- As telas (`TelaLista`, `TelaDetalhe`) não mudam: continuam recebendo valores e funções

Passo 11

O estado sai da tela: ViewModel com `StateFlow`

Antes: o estado dentro de `Conteudo`

```
```

- Funciona, mas as decisões (validar, gerar o id, limpar o texto) moram numa função de interface
- Testar `adicionar` exige montar a tela e clicar
- Não há onde iniciar uma busca que demora: o Passo 12 precisa de um escopo de corrotinas que não seja a composição

O ViewModel

```
```

As regras (`tituloValido`, `comNova`, `alternando`) são as funções puras da Sprint 1: o ViewModel só as chama.

O que fica no ViewModel e o que fica na tela

No ViewModel (StateFlow)	Na tela (remember / rememberSaveable)
a lista de tarefas	a tarefa selecionada no painel lateral
o texto do formulário	a posição de rolagem, o foco, animações
carregando, erro, vazio, sucesso	o que só muda a aparência
as regras: validar, gerar id, chamar o repositório	—

- A pergunta é: "isso é dado do app ou só da interface?"
- No exemplo, `selecionada` continua em `rememberSaveable` dentro de `Conteudo`: é estado só da interface
- O ViewModel não recebe `NavController`, `Context` nem tipos do Compose: só dados e o repositório

A tela lê o estado e chama eventos

Antes (Sprint 1)	Agora
`var tarefas by remember { mutableStateOf(...) }` na tela	`StateFlow` no ViewModel
as regras só se testam pela interface	o ViewModel se testa sozinho, sem tela

`collectAsStateWithLifecycle` para de coletar quando a tela sai de vista.

📖 **Ref.** Compose Multiplatform — ViewModel

Testar o ViewModel sem tela

Erros comuns no Passo 11

Erro	O que acontece	Como fica
`val estado = MutableStateFlow(...)` público	a tela consegue alterar o estado por fora	`_estado` privado e `asStateFlow()`
`TarefasViewModel()` direto no `@Composable`	um ViewModel novo a cada recomposição: o estado some	`viewModel { TarefasViewModel() }`
mudar uma `MutableList` dentro do estado	o objeto é o mesmo, e o `StateFlow` não emite	lista imutável e `copy`
lifecycle numa versão e Navigation em outra	"State must be at least CREATED" ao voltar de uma tela	versões da mesma família (2.9.x)
ler com `.value` dentro do `@Composable`	a tela não recompõe quando o estado muda	`collectAsStateWithLifecycle()`

Passo 12

Carregar sem travar a tela: estado selado

Os quatro estados como classe selada

```

```

```

```

- Classe selada: todos os casos são conhecidos na compilação. Um quinto estado faz o `when` deixar de compilar até alguém decidir como mostrá-lo
- Dentro do ramo `is EstadoTela.Erro`, `tela` já é um `Erro` (*smart cast*): `tela.mensagem` existe
- É o critério "Estados da interface" da rubrica: carregando, erro, vazio e sucesso

O estado da tela, com o conteúdo dentro

```
```

- O formulário existe em qualquer caso; a lista só existe no `Sucesso`
- `tarefas` é calculada a partir de `tela`: não há duas fontes para a mesma informação
- `as?` devolve `null` quando `tela` não é `Sucesso`; `orEmpty()` troca o `null` por lista vazia
- O estado começa em `Carregando`: a primeira coisa que a tela mostra é o indicador

Um repositório que demora de propósito

```
```

- `delay` suspende; `Thread.sleep` bloquearia a thread da interface
- É a porta que o vídeo vai implementar com a rede (Passo 14) e com cache (Passo 15)

Carregar numa corrotina

```
```

- O repositório em memória tem 800 ms de demora e um `falhar`: dá para ver os quatro estados sem rede
- `CancellationException` é como a corrotina é cancelada: capturá-la e seguir em frente impede o cancelamento

As telas de carregando e de erro

- O indicador tem descrição: o leitor de tela anuncia "Carregando tarefas"
- Toda tela de erro dá uma saída ao usuário; aqui, **Tentar de novo** chama `carregar()`
- O estado vazio é a `TelaLista` sem itens, com "Nenhuma tarefa ainda. Que tal criar a primeira?"

Demonstração

1. O app abre com o indicador de carregamento e, 800 ms depois, a lista
2. No `RepositorioEmMemoria`, `falhar = true`: aparece a mensagem e o botão **Tentar de novo**
3. Com a lista inicial vazia: aparece "Nenhuma tarefa ainda"

Testes com corrotinas

- O `viewModelScope` usa `Dispatchers.Main`, que não existe no teste: `setMain` põe um despachante de teste no lugar
- O repositório recebe `Duration.ZERO`: o teste não espera os 800 ms
- O teste só vê o estado final. Para conferir a sequência (carregando → sucesso), o vídeo traz o Turbine

Para conferir o entendimento

1. Por que `_estado` é privado e a tela recebe um `StateFlow`?
2. O que acontece se um `catch (e: Exception)` engolir a `CancellationException`?
3. Por que o `when` sobre `EstadoTela` não tem `else`?
4. Onde fica a tarefa selecionada no painel lateral, e por quê?
5. Num teste, por que o ViewModel recebe um `RepositorioEmMemoria(Duration.ZERO, ...)`?

No vídeo (até 30/10)

Passo	Assunto
13	Camadas (domínio, dados, apresentação) e injeção de dependência com Koin
14	Dados reais: Ktor Client, DTO e erros de rede com nome, chamando a API de Web II
15	Abrir sem rede: a última lista guardada com DataStore
16	Testes de ViewModel com Turbine e dublês

Até lá, no projeto do grupo:

- levar o estado de uma tela para um ViewModel com `StateFlow`
- modelar os estados dessa tela como classe selada, com os quatro casos
- escolher a API real que o app vai consumir

Onde estudar depois

- Leitura da sprint: `leituras/moveis-s2.md`, capítulos 1 a 4
- Passos: `exemplos/tarefas-compose/PASSOS.md` · branch `aula`, tags `passo-11` e `passo-12`
- Android — Kotlin coroutines on Android
- Android — StateFlow and SharedFlow
- Compose Multiplatform — ViewModel
- Kotlin — Sealed classes · Coroutines basics